

BilgiFit: Gym Reservation System

CMPE 372.02 — User Interface Design · Full Submission Report

Group 3

Ece Doğa Gül (121200123) · Berkay Özdelice (119200057) · Baki Kerem Artan (121200154)

Selim Şengil (122200085) · Cem Akçiçek (122200044)

Istanbul Bilgi University · June 2026

Table of Contents

Project Overview.....	3
Personas.....	4
Persona 1 – Student User.....	4
Persona 2 – Instructor User.....	4
Persona 3 – Administrator User.....	5
User Stories.....	5
Student User Stories (Zeynep).....	5
Instructor User Stories (Emre).....	6
Shared & Administrator User Stories.....	6
User Journey.....	6
Student User Journey Example.....	6
User Journey Map.....	6
Information Architecture (IA).....	7
IA Goals.....	7
Global Navigation Model.....	8
Sitemap (Page Hierarchy).....	8
Core Navigation Paths (Task Flows).....	8
Labeling System.....	9

Page Content Inventory.....	10
Instructor Panel.....	10
Admin Dashboard.....	10
Wireframes.....	11
Low Fidelity Screens.....	11
High Fidelity Screens.....	11
Design System.....	15
Color System.....	15
Typography.....	15
Spacing & Grid System.....	15
Core UI Components.....	16
States & Microinteractions.....	16
ER Data Model.....	16
Relational Schema.....	18
Business Rules (ER Constraints).....	18
Relationships & Cardinality.....	19
CRUD Matrix.....	19
SQL Tables.....	20
Relational Schema Design: Tables + Primary Key / Foreign Key + Normalization.....	20
Relationships in Relational Form (PK/FK Summary).....	22
Integrity Constraints (Must-have).....	22
Sample SQL Logic.....	23
API Endpoints.....	23
Async Fetch Flow.....	24
Accessibility Review.....	25
Final Reflection.....	26

Live demo: <https://cmpe372-bilgifit-1.onrender.com/>

Source code: <https://github.com/Becho-png/CMPE372-Bilgifit>

Figma prototype:

<https://www.figma.com/design/rs4aRJcavArfGf8ne2doTo/BilgiFit-Assignment-3—High-Fidelity-UI?node-id=1-852>

Project Overview

This project designs and implements a data-driven gym reservation web application intended for use within a university campus sports center. The system provides a centralized digital platform that enables students, instructors, and administrators to reserve facilities, join group sessions, book personal trainers, and manage reservations efficiently.

Traditional campus facility access is often unstructured: users arrive at the gym only to find it overcrowded, a court already in use, or a class full. These approaches lead to wasted time, reservation conflicts, and frustration, and they make it difficult for instructors and administrators to coordinate sessions and capacity. The proposed system addresses these issues by offering a structured, real-time, and accessible web-based solution with role-based interactions.

The application allows:

- **Students / members** to view real-time availability, reserve facility slots, join group sessions, book personal trainers, and track their reservations.
- **Instructors** to reserve facilities for teaching and to create instructor-led group training sessions.
- **Administrators** to monitor reservation activity, occupancy, and capacity compliance.

From a UI/UX perspective, the project emphasizes user-centered design principles, clear information architecture, minimal cognitive load, and accessibility and usability across devices.

From a technical perspective, the system is designed as a data-driven web application, where user interfaces map directly to underlying data models, forms correspond to database entities, CRUD (Create, Read, Update, Delete) operations support core workflows, and asynchronous data interactions enable responsive user experiences.

The project follows an iterative design and validation process, beginning with user analysis (personas, user stories, journey maps), progressing through low- and high-fidelity prototyping, and culminating in an implementation-ready, deployed full-stack application with a relational database, CRUD

matrices, and an asynchronous API. The final outcome is a scalable, maintainable gym reservation system that demonstrates how thoughtful UI/UX design combined with sound system architecture delivers real organizational value.

Personas

Personas were developed to represent the primary user groups of the BilgiFit system and to guide interface, functionality, and data-modeling decisions. Three main personas were identified based on typical campus stakeholders: students, instructors, and administrators.

Persona 1 – Student User

Field	Detail
Name	Zeynep Kaya
Age	21
Role	Active gym user (Computer Engineering student)
Technology comfort	High
Goals	Reserve facilities quickly, view real-time availability, join classes, book a personal trainer
Frustrations	Arriving to an overcrowded gym, losing slots before booking, not knowing which facility is free
Typical tasks	Logging in, checking gym occupancy, reserving a basketball court, joining a group session, cancelling when plans change
System needs	Simple reservation flow, clear capacity info, mobile-friendly UI, confirmation feedback

Persona 2 – Instructor User

Field	Detail
Name	Emre Yıldırım
Age	29
Role	Fitness instructor
Technology comfort	Medium
Goals	Reserve facilities for lessons, create group sessions, manage participants, avoid conflicts
Frustrations	Manual coordination, unclear reservations, last-minute facility conflicts, no view of existing sessions

Field	Detail
Typical tasks	Reserving the boxing ring for training, creating a group class, checking participant counts, reviewing upcoming sessions
System needs	Reliable facility reservation, session creation tools, participant tracking, conflict prevention

Persona 3 – Administrator User

Field	Detail
Name	Derya Acar
Age	35
Role	Sports center administrator
Technology comfort	Medium
Goals	Overview of facility usage, reservation activity, occupancy levels, and peak hours
Frustrations	Lack of visibility into daily usage, conflicts from overlapping reservations, manual monitoring
Typical tasks	Monitoring reservations, reviewing usage statistics, checking capacity compliance, resolving issues
System needs	Monitoring dashboard, usage statistics, capacity reporting, role-based permissions

These personas guided navigation structure, form design, database entity design, CRUD workflow definitions, and access-control policies.

User Stories

Student User Stories (Zeynep)

ID	User Story
US1	As a student, I want to reserve a gym time slot, so that I can work out without overcrowding.
US2	As a student, I want to reserve the basketball court, so that I can play at a scheduled time.
US3	As a student, I want to reserve the boxing ring, so that I can train properly.
US4	As a student, I want to see facility availability, so that I can choose the best time.
US5	As a student, I want to book a personal trainer, so that I can get one-on-one training.
US6	As a student, I want to join a group training

ID	User Story
	session, so that I can train with others.

Instructor User Stories (Emre)

ID	User Story
US7	As an instructor, I want to reserve facilities, so that I can conduct training sessions.
US8	As an instructor, I want to create group classes, so that multiple users can join my sessions.

Shared & Administrator User Stories

ID	User Story
US9	As a user, I want to receive confirmation after booking, so that I know the action succeeded.
US10	As a user, I want to cancel my reservation, so that I can free my slot if my plans change.
US11	As a user, I want the system to prevent double-booking, so that schedules remain accurate.
US12	As an administrator, I want to monitor reservations, so that I can manage facility usage efficiently.

User Journey

Student User Journey Example

Step	Action	System Response
1	User logs in	Authenticated, redirected to Dashboard
2	User opens Facility Booking	Facility slots and capacity displayed
3	User selects an available slot	Confirmation modal shown
4	User confirms reservation	Slot booked, capacity updated
5	User views Dashboard	Reservation appears in "Upcoming reservations"

User Journey Map

Persona: Student (Zeynep) · **Scenario:** discovers, joins, and attends a group training session.

Stage	Actions	Emotions	Pain Points	Opportunities
1. Login	Opens the app and logs in	Neutral, routine	Forgetting password slows the user down	Quick login with a remember-me option
2. Browse sessions	Views instructor-created classes	Curious, interested	Hard to find relevant sessions in a long list	Filters by facility type, instructor, or time
3. View details	Sees instructor, facility, time, open spots	Excited, evaluating	Not enough info about the session	Clear session cards with key details upfront
4. Join	Selects a session and confirms	Motivated, satisfied	Worries the session may fill before confirming	Real-time spot counter + instant confirmation
5. Confirmation	Receives a confirmation message	Reassured, confident	May forget the session time later	Confirmation summary + dashboard reminder
6. Pre-session check	Double-checks time/facility on dashboard	Calm, prepared	Dashboard may feel cluttered	Clean upcoming-sessions section
7. Attending	Arrives at the correct facility on time	Energized, ready	Anxiety if the room is overcrowded	Occupancy display reassures capacity is respected
8. Post-session	Considers booking again or cancelling	Satisfied, reflective	Cancellation process may not be obvious	Prominent cancel button with deadline reminder

Information Architecture (IA)

IA Goals

In this project, the information architecture is designed to:

- Reduce cognitive load by grouping tasks into clear modules (Dashboard, Facility Booking, Group Sessions, Personal Trainers, Profile).
- Support role-based navigation (Student / Instructor / Administrator) without hiding core discovery.
- Enable a fast reservation workflow: Browse → Select → Confirm → Manage.
- Ensure consistency across pages through predictable placement of navigation, filters, and actions.

Global Navigation Model

Primary navigation (persistent left sidebar), common to all roles: Dashboard, Facility Booking, Group Sessions, Personal Trainers, My Profile. Role-specific entry (visible by permission): Instructor Panel (for instructors); Admin Dashboard (for administrators). Secondary navigation (within modules): facility-type filters (All / Gym / Boxing / Basketball), session level badges, and modal confirmation steps.

Sitemap (Page Hierarchy)

Public area (no login required): Home / Login-Register.

Authenticated area (Student / Instructor): Dashboard (next reservation, KPIs, quick actions, upcoming reservations); Facility Booking (filters, capacity bars, reserve action); Group Sessions (session cards, join action); Personal Trainers (trainer cards, book action); My Profile (contact info, membership, reservations).

Instructor Panel (instructors): Create Session; Manage Sessions.

Admin Dashboard (administrators): Facility Usage; Reservation Monitoring.

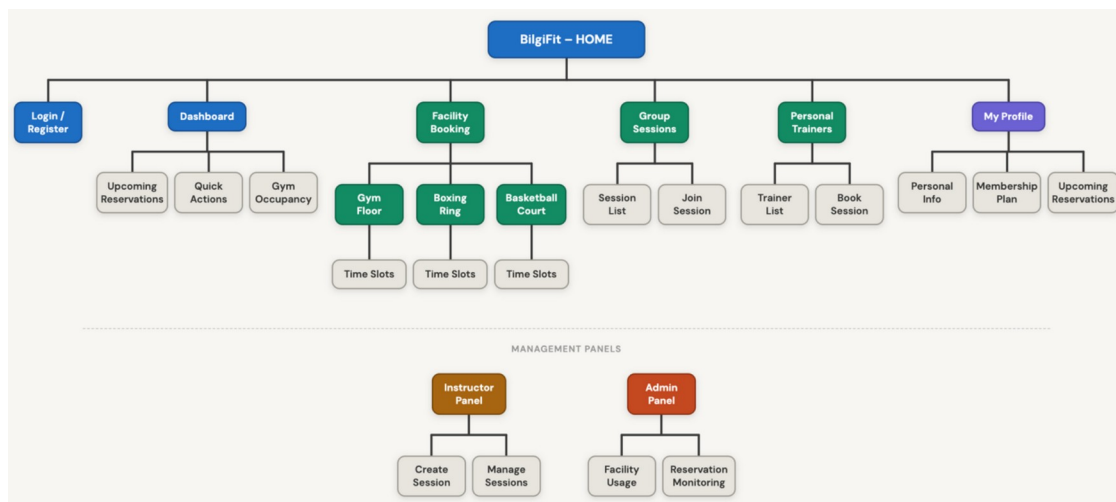


Figure 1. BilgiFit sitemap — the main navigation hierarchy, user-facing pages, and role-specific management panels.

Core Navigation Paths (Task Flows)

A. Facility Reservation Flow: Home → Login → Dashboard → Facility Booking → Filter → Select Slot → Confirm Modal → Confirmation → My Reservations.

B. Group Session Flow: Dashboard → Group Sessions → Select Session → Join → Joined State → Dashboard.

C. Personal Trainer Flow: Dashboard → Personal Trainers → Select Trainer → Book → Booked State → Dashboard.

D. Cancellation Flow: Dashboard → Upcoming Reservations → Cancel → Status updated to “Cancelled”.

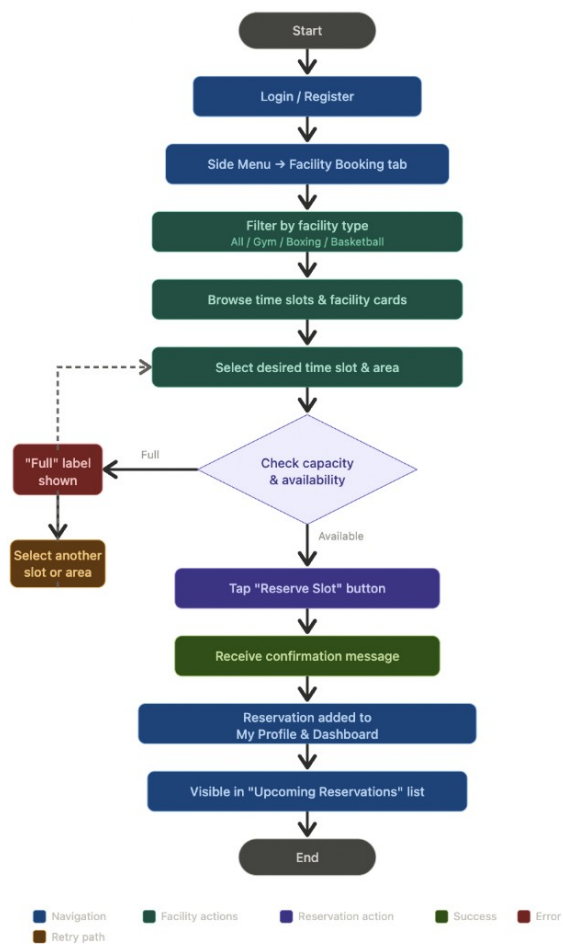


Figure 2. Task flow for the main scenario — reserving a facility — including the capacity check, the full-capacity branch, and the final confirmation outcome.

Labeling System

To prevent confusion, labels are consistent and user-friendly: Reserve Slot (not “Create record”); Join Session (not “Insert participant”); Book Session (not “Initiate request”); My Profile / My Reservations. Status labels: Available, Full, Active, Pending, Cancelled, Unavailable. Facility types: Gym, Boxing, Basketball. Session levels: Beginner, Intermediate, Advanced, All Levels.

Page Content Inventory

Page	Purpose	Main content	User actions
Dashboard	Overview	Next reservation, bookings-this-week, gym occupancy, open courts, upcoming-reservations table, quick actions	Navigate, cancel reservation
Facility Booking	Reserve slots	Filter tabs, facility cards (name, time, capacity bar, status badge)	Filter, reserve, view full state
Group Sessions	Join classes	Session cards (name, instructor, schedule, level badge, participant count)	Join session
Personal Trainers	Book PT	Trainer cards (avatar, specialty, rating, session count, availability)	Book session
My Profile	Manage account	Editable name/email/phone/address, membership card	Edit & save profile

Instructor Panel

Page	Purpose	Content	Actions
Create Session	Add group sessions	Session form (name, schedule, capacity, level)	Submit session
Manage Sessions	Manage own sessions	List of created sessions, participant counts	Edit / review

Admin Dashboard

Page	Purpose	Content	Actions
Facility Usage	Monitor system	Occupancy KPIs, capacity compliance	Navigate / review
Reservation Monitoring	Oversee bookings	Reservation list, statuses	Review / export

Wireframes

Low Fidelity Screens

Early digital low-fidelity sketches established layout structure, content blocks, button placement, and navigation relationships before visual styling: the homepage (clear search + categories), the facility list (card design + filters), and the detail/booking page (hierarchy + primary CTA button).

High Fidelity Screens

The Figma file contains eight connected high-fidelity screens demonstrating consistent visual hierarchy, real components (cards, badges, capacity bars), and clickable navigation that simulates a real reservation flow. The seven desktop screens are shown below; the implemented HTML/CSS/JS screens match these designs.

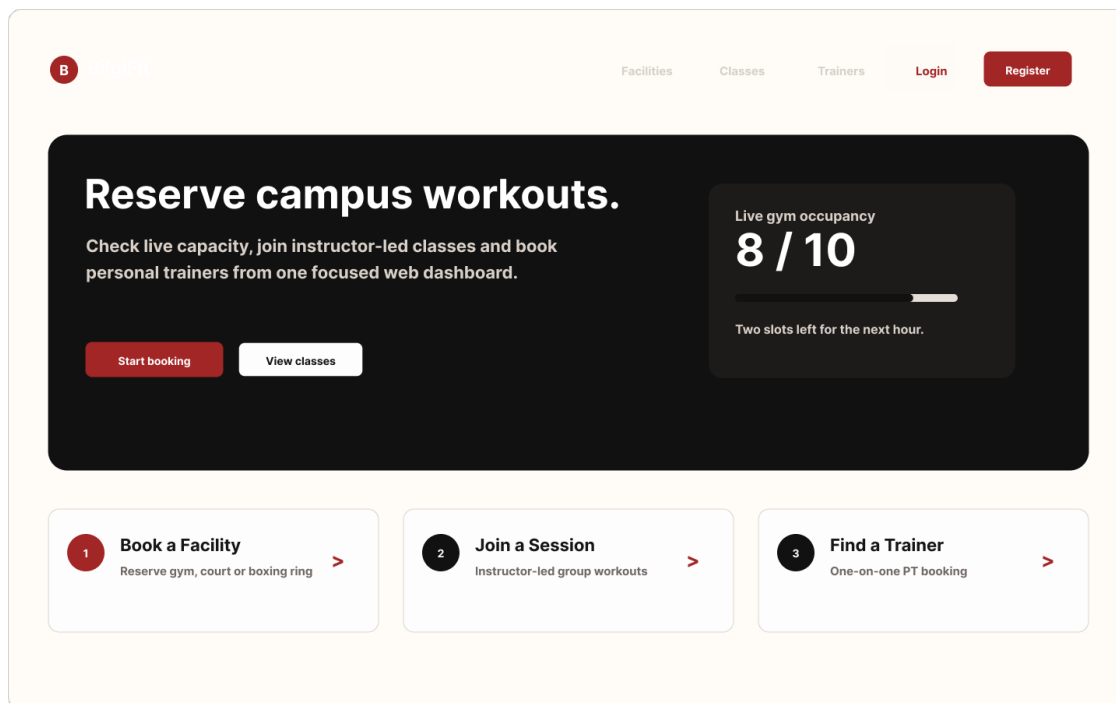


Figure 3. Homepage — value proposition, live gym occupancy, and three entry points (Book a Facility, Join a Session, Find a Trainer).

BilgiFit

Welcome back to BilgiFit.

Use your Bilgi account to manage reservations, classes and trainer sessions.

Next available gym slot
Today / 17:00

Log in

Continue with your campus credentials.

[Login](#) [Register](#)

Email
zeynep@bilgi.edu.tr

Password

[Log in](#)

[Forgot password?](#)

Figure 4. Login / Register — split layout with a dark brand panel and a form using Bilgi-email validation.

BilgiFit
campus sports center

[Dashboard](#)
[Facility Booking](#)
[Group Sessions](#)
[Personal Trainers](#)
[My Profile](#)

Live occupancy
8 / 10

Dashboard

Overview of your reservations and live facility state

Search facilities...

Next reservation
Pilates Core
Today / 18:00 / Studio A [View details](#)

4
Bookings this week

8/10
Gym occupancy

2
Open courts

Upcoming reservations

Facility	Date	Time	Status
Gym Floor	Today	17:00	Active
Basketball Court	Thu	19:00	Active
PT Deniz	Fri	12:00	Pending

Quick actions

Reserve facility [>](#)
Book available gym/court time

Join class [>](#)
Browse group sessions

Book trainer [>](#)
Find one-on-one support

Figure 5. Dashboard — next reservation, weekly bookings, live gym occupancy, quick actions, and the upcoming-reservations table.

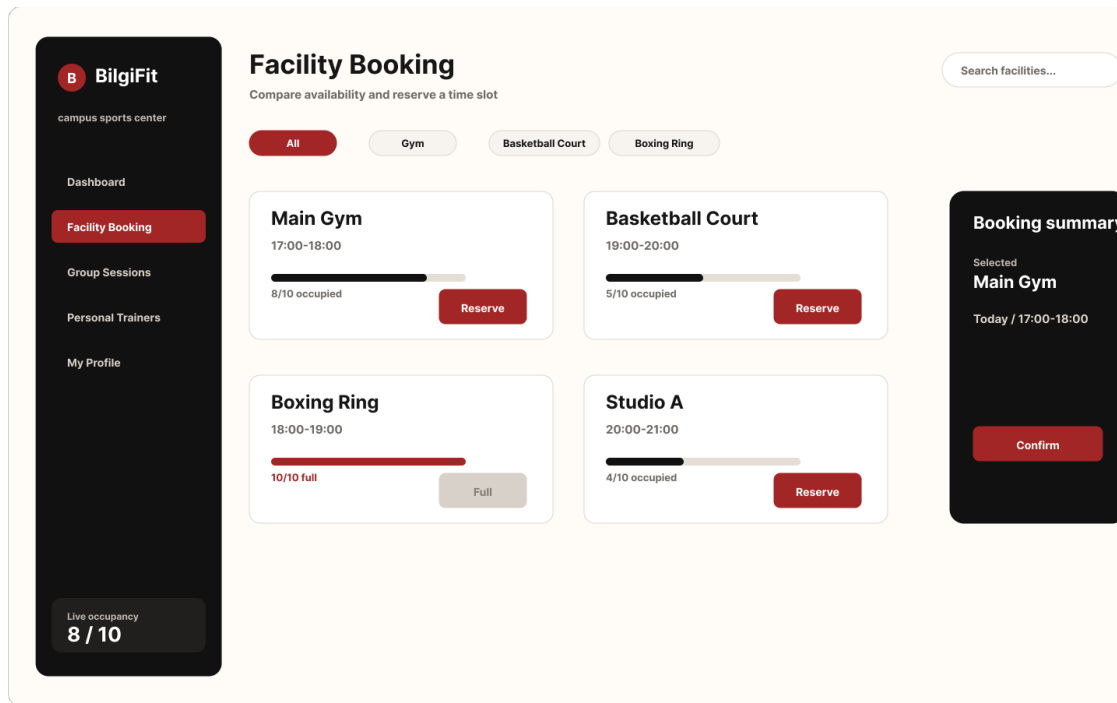


Figure 6. Facility Booking — facility-type filters, per-slot capacity bars, and Reserve / Full button states.

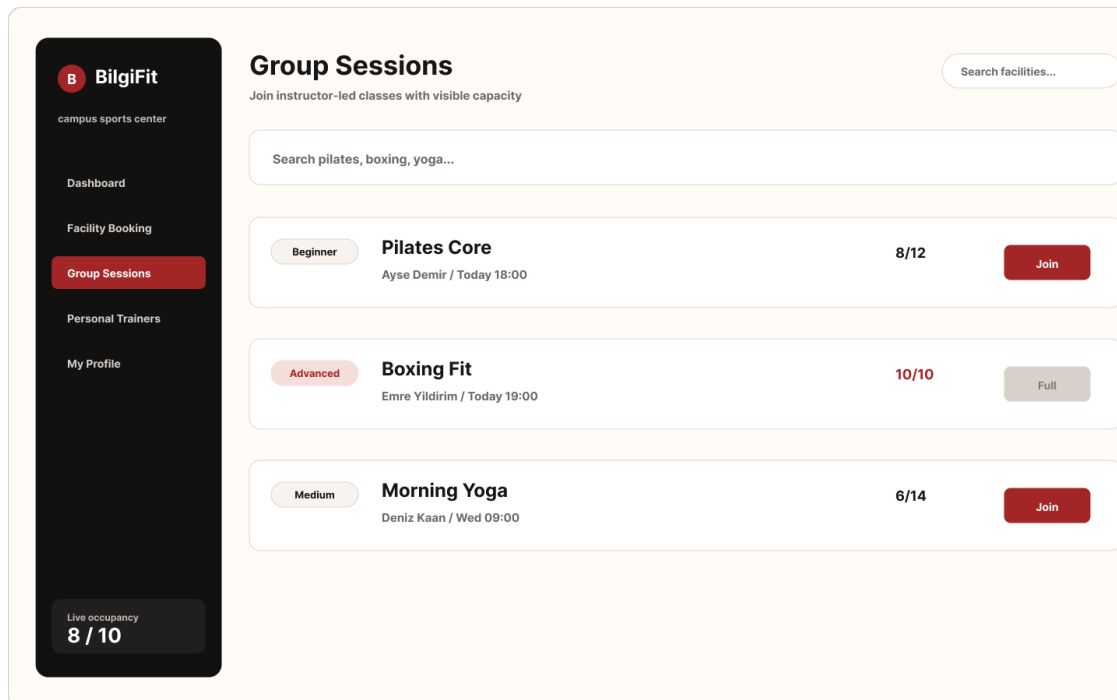


Figure 7. Group Sessions — session cards with level badges, participant counts, and Join / Full states.

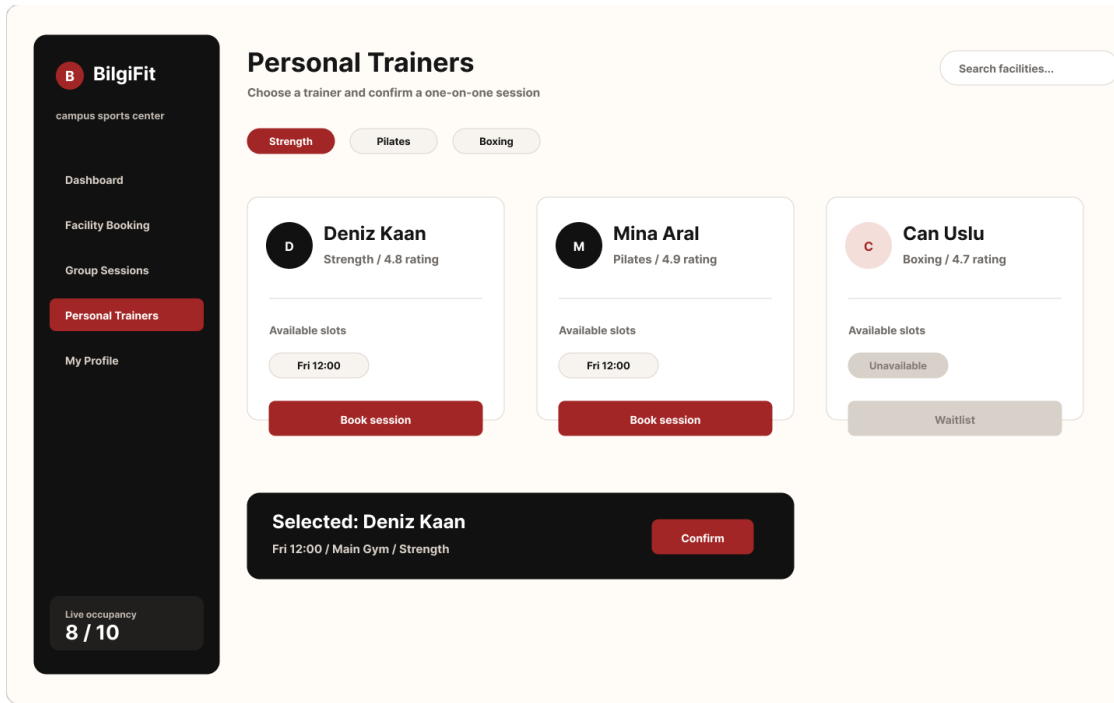


Figure 8. Personal Trainers — trainer cards with specialties, ratings, session counts, and Book / Unavailable states.

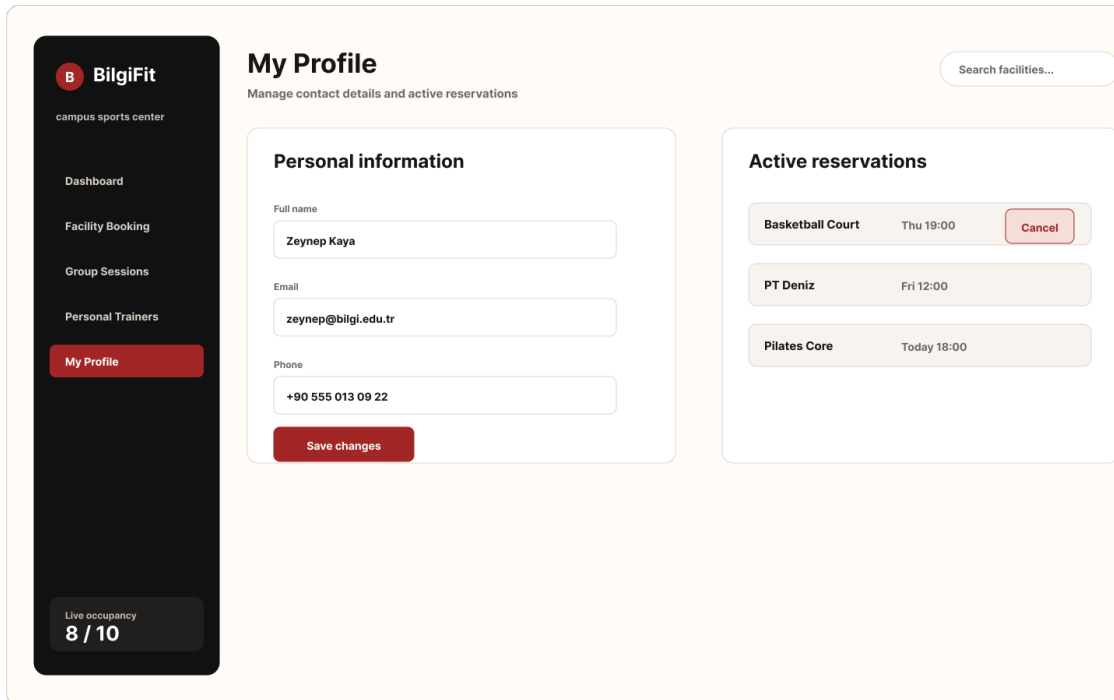


Figure 9. My Profile — personal information, editable fields, and membership status.

Design System

This section documents the unified design system created for the BilgiFit web application. The design system ensures visual consistency, usability, accessibility, and scalability across all screens. The interface uses a red, charcoal-black, and warm-white identity that signals energy and action while keeping core workflow screens calm and readable.

Color System

Purpose	Color	Usage
Primary action	Red	Log in, Reserve, Join, Book session, Confirm; active sidebar item
Surface (dark)	Charcoal / black	Sidebar, brand area, hero panels, primary text
Background	Warm white / light gray	Page background and cards
Success	Green	Confirmation toasts and “Active” badges
Warning / pending	Amber	“Pending” badges and full-capacity indication
Error	Red border + soft red fill	Invalid inputs and form-level alerts

Colors were selected to provide sufficient contrast ratios to meet WCAG 2.1 AA accessibility requirements.

Typography

Usage	Font	Size
Headings (H1)	Inter / sans-serif	34-58 px, bold
Section / card title	Inter / sans-serif	22-34 px, bold
Body text	Inter / sans-serif	14-17 px, medium
Labels & badges	Inter / sans-serif	12-14 px, bold

Typography supports clear hierarchy, mobile readability, and accessibility compliance.

Spacing & Grid System

Desktop frame 1440 × 900 px; frame inner padding 36-52 px; sidebar width ~238 px (consistent across screens); card gaps 24-32 px; button height 44 px and input height 48 px; card radius 8-18 px; base spacing rhythm 8 px.

Core UI Components

Component	Description
Buttons	Primary (red), Secondary (outline), Disabled, Danger
Input fields	Text, email, tel, dropdown
Cards	Facility / session / trainer preview and result display
Badges	Status indicators (Available, Full, Active, Pending, Unavailable)
Modal	Reservation confirmation
Navigation bar	Persistent sidebar (global navigation)
Capacity bar	Visual progress indicator of slot/session occupancy
Toast	Transient success/error feedback

States & Microinteractions

State	Behavior
Hover	Highlight border & shadow
Focus	Visible outline (keyboard accessibility)
Loading	Temporary state during async calls
Error	Red border & helper text
Success	Green confirmation toast

Capacity is communicated three ways at once — progress bar, count text, and a Full/Available badge — so meaning never depends on color alone. Microinteractions provide immediate system feedback and improve perceived usability.

ER Data Model

The system is built on seven core entities (the model deployed in the live PostgreSQL database).

Entity 1: users — all registered members/students. Key attributes: id (PK), fullname, email (unique), password (hash).

Entity 2: facility_slots — bookable facility time slots. Key attributes: id (PK), facility_name, facility_type (gym/boxing/basketball), slot_time, capacity, booked.

Entity 3: group_sessions — instructor-led classes. Key attributes: id (PK), session_name, instructor, schedule, participant_count, max_capacity, level.

Entity 4: personal_trainers — trainer profiles. Key attributes: id (PK), trainer_name, specialty, rating, session_count, available.

Entity 5: reservations — unified booking log (facility / session / trainer). Key attributes: id (PK), user_id (FK → users), reservation_title, reservation_date, reservation_time, status.

Entity 6: session_participants — junction entity resolving the many-to-many relationship between users and group sessions. Key attributes: id (PK), user_id (FK → users), session_id (FK → group_sessions).

Entity 7: pt_bookings — one-on-one trainer bookings. Key attributes: id (PK), user_id (FK → users), trainer_id (FK → personal_trainers), booking_date, booking_time, status.

The conceptual ER model (Assignment 2) is shown in Figure 10. It names the same seven entities using domain terms (User, Facility, Reservation, GroupSession, PersonalTrainer, PTBooking, SessionParticipant); the deployed implementation keeps these entities and relationships but uses database-friendly names and a single unified reservations log for the dashboard view. The SQL Tables section documents the implemented schema in database/schema.sql.

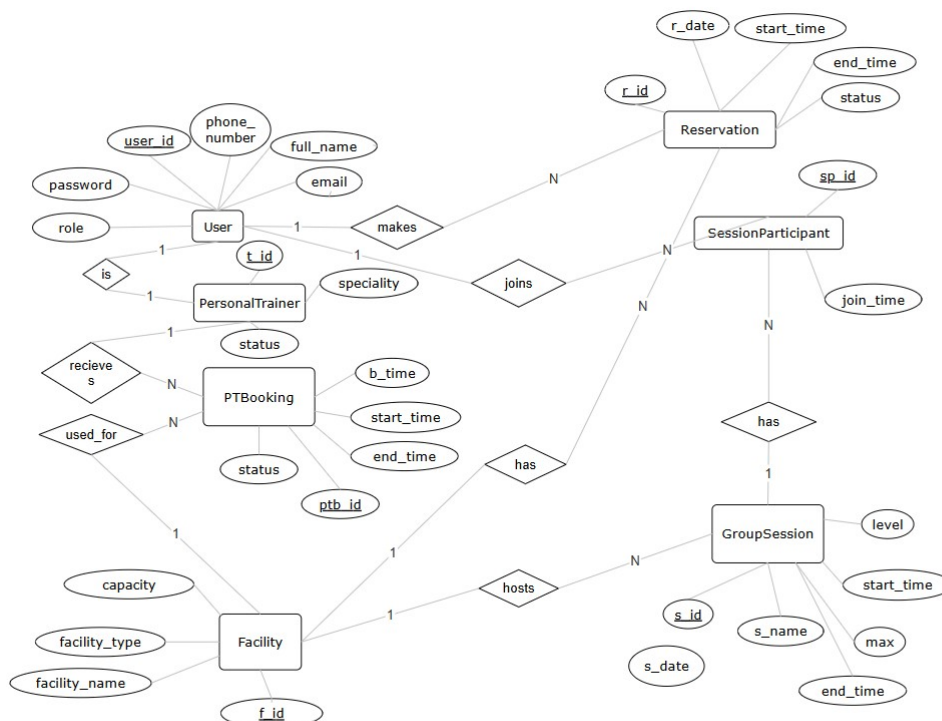


Figure 10. Conceptual Entity-Relationship diagram (Chen notation) showing the seven entities, their attributes, relationships, and cardinalities.

Relational Schema

The ER model maps to seven relational tables. Figure 11 shows the conceptual relational schema with primary keys and foreign-key relationships.

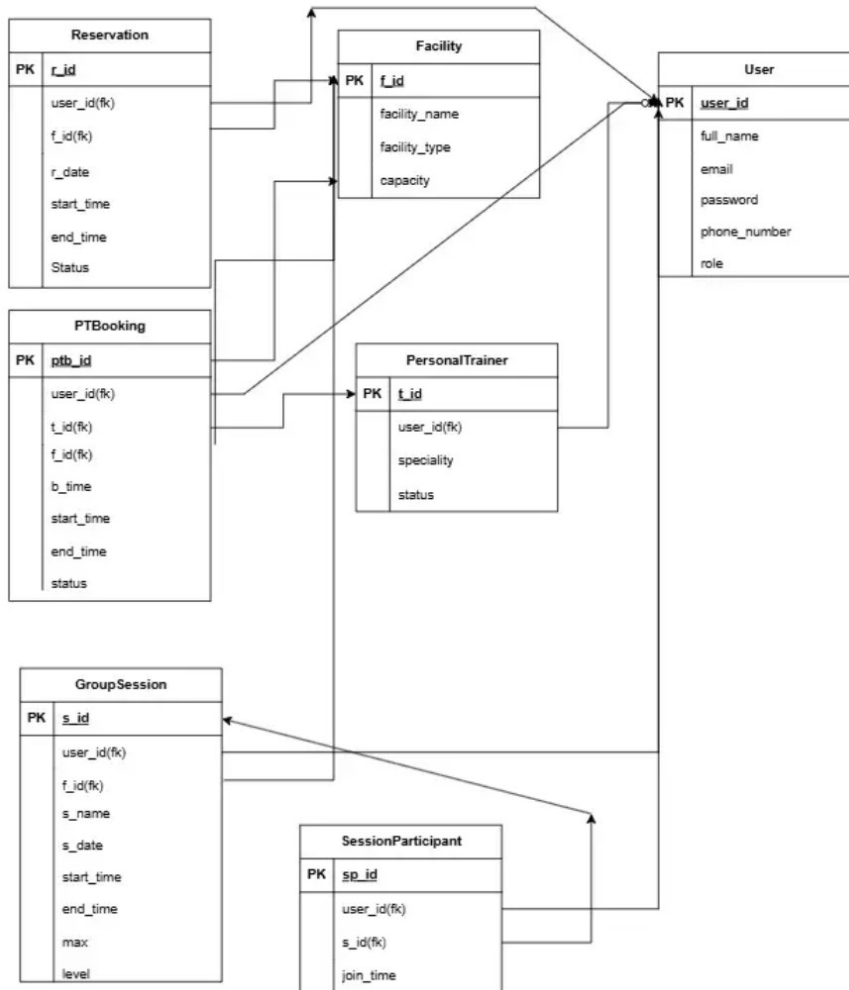


Figure 11. Relational schema — tables with primary keys (PK) and foreign-key (FK) relationships derived from the ER model.

Business Rules (ER Constraints)

1. Only @bilgi.edu.tr and @bilgiedu.net email domains are accepted.
2. Each email is unique — one account per address.
3. The gym area cannot exceed its capacity (10) per time slot; booked $\leq$ capacity always holds.
4. A facility cannot be reserved when booked $\geq$ capacity ("Full").
5. A user cannot join the same group session twice.
6. A user cannot create duplicate bookings for the same trainer.

7. A full facility/session remains visible but disabled rather than hidden.
8. Cancellation is a soft delete (status = 'Cancelled') to preserve reservation history.
9. Passwords are stored only as Werkzeug hashes, never plaintext.

Relationships & Cardinality

- **R1: users — create — reservations.** User (1) → Reservation (N).
- **R2: facility_slots — appear in — reservations.** Facility (1) → Reservation (N).
- **R3: users — join — group_sessions (via session_participants).** User (N) ↔ GroupSession (M).
- **R4: users — make — pt_bookings.** User (1) → PTBooking (N).
- **R5: personal_trainers — receive — pt_bookings.** Trainer (1) → PTBooking (N).
- **R6: users (instructor) — create — group_sessions.** User (1) → GroupSession (N).

CRUD Matrix

UI screen / action	users	facility_slots	group_sessions	personal_trainers	reservations	session_participants	pt_bookings
Register	C	-	-	-	-	-	-
Login	R	-	-	-	-	-	-
Dashboard (load)	-	R	R	R	R	-	-
Facility Booking (browse)	-	R	-	-	-	-	-
Reserve facility	-	U	-	-	C	-	-
Group Sessions (browse)	-	-	R	-	-	-	-
Join session	-	-	U	-	C	C	-
Personal Trainers (browse)	-	-	-	R	-	-	-
Book trainer	-	-	-	U	C	-	C
Cancel reservation	-	-	-	-	U (soft delete)	-	-

UI screen / action	users	facility_slots	group_sessions	personal_trainers	reservations	session_participants	pt_bookings
My Profile (save)	U	-	-	-	-	-	-
Admin monitoring	-	R	R	R	R	R	R

SQL Tables

Relational Schema Design: Tables + Primary Key / Foreign Key + Normalization

The full DDL is provided in database/schema.sql. Each table is in Third Normal Form.

users

Column	Type	Constraints
id	SERIAL	PK
fullname	VARCHAR(120)	NOT NULL
email	VARCHAR(160)	NOT NULL, UNIQUE, CHECK (Bilgi domain)
password	VARCHAR(255)	NOT NULL (Werkzeug hash)

facility_slots

Column	Type	Constraints
id	SERIAL	PK
facility_name	VARCHAR(120)	NOT NULL
facility_type	VARCHAR(40)	NOT NULL, CHECK IN ('gym', 'boxing', 'basketball')
slot_time	VARCHAR(40)	NOT NULL
capacity	INTEGER	NOT NULL, CHECK (capacity > 0)
booked	INTEGER	NOT NULL DEFAULT 0, CHECK (booked ≤ capacity)

group_sessions

Column	Type	Constraints
id	SERIAL	PK

Column	Type	Constraints
session_name	VARCHAR(120)	NOT NULL
instructor	VARCHAR(120)	NOT NULL
schedule	VARCHAR(80)	NOT NULL
participant_count	INTEGER	NOT NULL DEFAULT 0
max_capacity	INTEGER	NOT NULL, CHECK (> 0)
level	VARCHAR(40)	NOT NULL, CHECK IN ('Beginner', 'Intermediate', 'Advanced', 'All Levels')

personal_trainers

Column	Type	Constraints
id	SERIAL	PK
trainer_name	VARCHAR(120)	NOT NULL
specialty	VARCHAR(120)	NOT NULL
rating	NUMERIC(2,1)	CHECK (0-5)
session_count	INTEGER	NOT NULL DEFAULT 0
available	BOOLEAN	NOT NULL DEFAULT TRUE

reservations

Column	Type	Constraints
id	SERIAL	PK
user_id	INTEGER	FK → users(id), NOT NULL, ON DELETE CASCADE
reservation_title	VARCHAR(160)	NOT NULL
reservation_date	VARCHAR(40)	NOT NULL
reservation_time	VARCHAR(40)	NOT NULL
status	VARCHAR(20)	NOT NULL DEFAULT 'Active', CHECK IN ('Active', 'Pending', 'Cancelled')

session_participants

Column	Type	Constraints
id	SERIAL	PK
user_id	INTEGER	FK → users(id), NOT NULL, ON DELETE CASCADE
session_id	INTEGER	FK → group_sessions(id), NOT NULL, ON DELETE CASCADE

Column	Type	Constraints
(composite)	-	UNIQUE (user_id, session_id)

pt_bookings

Column	Type	Constraints
id	SERIAL	PK
user_id	INTEGER	FK → users(id), NOT NULL, ON DELETE CASCADE
trainer_id	INTEGER	FK → personal_trainers(id), NOT NULL, ON DELETE RESTRICT
booking_date	VARCHAR(40)	NOT NULL
booking_time	VARCHAR(40)	NOT NULL
status	VARCHAR(20)	NOT NULL DEFAULT 'Pending', CHECK IN ('Pending', 'Confirmed', 'Cancelled')
(composite)	-	UNIQUE (user_id, trainer_id)

Normalization. *1NF* — every attribute is atomic and every table has a primary key; there are no repeating groups. *2NF* — all tables use single-column primary keys, so there are no partial dependencies. *3NF* — no transitive dependencies: trainer details live only in personal_trainers, facility details only in facility_slots, and reservations reference users by user_id instead of duplicating member data.

Relationships in Relational Form (PK/FK Summary)

- reservations.user_id → users.id (mandatory)
- session_participants.user_id → users.id (mandatory)
- session_participants.session_id → group_sessions.id (mandatory)
- pt_bookings.user_id → users.id (mandatory)
- pt_bookings.trainer_id → personal_trainers.id (mandatory)

Integrity Constraints (Must-have)

- users.email UNIQUE + Bilgi-domain CHECK
- facility_slots: facility_type CHECK; capacity > 0; booked ≤ capacity
- group_sessions: participant_count ≤ max_capacity; level CHECK
- reservations.status CHECK ('Active', 'Pending', 'Cancelled')
- pt_bookings.status CHECK ('Pending', 'Confirmed', 'Cancelled')
- UNIQUE (user_id, session_id) on session_participants (no duplicate joins)
- UNIQUE (user_id, trainer_id) on pt_bookings (no duplicate bookings)

- Delete behavior: reservations / session_participants / pt_bookings → CASCADE on user; pt_bookings.trainer_id → RESTRICT

Sample SQL Logic

Representative queries (full set in database/crud_queries.sql):

```
-- READ: authenticate a user (POST /login)
SELECT id, fullname, email, password FROM users WHERE email = $1;

-- READ: facility slots for the booking screen (GET /api/facilities)
SELECT id, facility_name, facility_type, slot_time, capacity, booked
FROM facility_slots ORDER BY id;

-- CREATE + UPDATE: reserve a facility (POST /api/reserve-facility)
UPDATE facility_slots SET booked = booked + 1 WHERE id = $1;
INSERT INTO reservations (user_id, reservation_title,
reservation_date, reservation_time, status)
VALUES ($1, $2, 'Today', $3, 'Active');

-- UPDATE: soft-delete cancellation (POST /api/cancel-reservation)
UPDATE reservations SET status = 'Cancelled' WHERE id = $1;

-- JOIN: trainer booking volume (admin report)
SELECT pt.trainer_name, COUNT(b.id) AS total_bookings
FROM personal_trainers pt
LEFT JOIN pt_bookings b ON b.trainer_id = pt.id
GROUP BY pt.id, pt.trainer_name ORDER BY total_bookings DESC;
```

API Endpoints

The Flask backend exposes a REST API consumed by the frontend via fetch.

Method	Endpoint	Purpose
POST	/register	Create account (Bilgi-email + hashed password)
POST	/login	Authenticate, return user object
GET	/api/facilities	List facility slots
POST	/api/reserve-facility	Reserve a slot (capacity-checked)
GET	/api/sessions	List group sessions
POST	/api/join-session	Join a session (capacity + duplicate checked)
GET	/api/trainers	List personal trainers
POST	/api/book-trainer	Request a trainer booking

Method	Endpoint	Purpose
GET	/api/reservations	List reservations (optional ? user_id=)
POST	/api/cancel-reservation	Soft-cancel a reservation
GET	/api/stats	Dashboard KPIs (gym occupancy, open courts, weekly bookings)

Responses are JSON with a success flag and a human-readable message, using correct HTTP status codes (201, 400, 401, 404, 409, 500).

Async Fetch Flow

This section describes how the BilgiFit web application retrieves, updates, and synchronizes data between the user interface and the database using asynchronous communication. Pages do not reload while data is loading; users get immediate feedback; multiple users can interact simultaneously; and the UI stays responsive during network delays.

BilgiFit — Asynchronous Client-Server Architecture

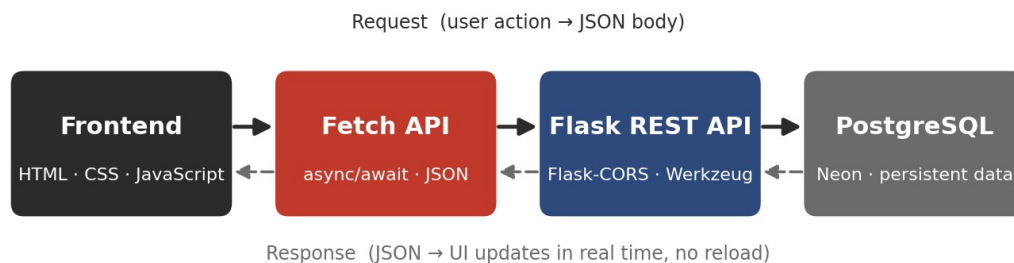


Figure 12. System architecture — the asynchronous client-server data flow between the frontend, the Fetch API layer, the Flask REST API, and the PostgreSQL database.

Core fetch architecture:

Layer	Technology	Responsibility
Frontend	HTML / CSS / JavaScript	UI rendering & user interactions
Communication	Fetch API (async/await)	Sends & receives JSON
Backend	Flask (Flask-CORS, Werkzeug)	Processes logic, validation, hashing
Database	PostgreSQL (Neon)	Stores permanent data
Deployment	Render + Gunicorn	Hosts the live application

Example async flow — reserving a facility (static/dashboard.js):

```
const response = await fetch("/api/reserve-facility", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ user_id: user.id, facility_id:
pendingFacilityId })
});
const data = await response.json();
if (!response.ok) { showToast(data.message); return; }
showToast("Reservation confirmed.");
closeModal();
await loadDashboardData(); // re-fetch + re-render without a page
reload
```

On the server, the corresponding endpoint validates capacity, then updates and inserts atomically:

```
UPDATE facility_slots SET booked = booked + 1 WHERE id = $1;
INSERT INTO reservations (user_id, reservation_title,
reservation_date, reservation_time, status)
VALUES ($1, $2, 'Today', $3, 'Active');
```

Dynamic content & validation. The dashboard loads facilities, sessions, trainers, and reservations concurrently with `Promise.all(...)`, then renders capacity bars, participant counts, and Full/Available/Joined/Booked button states from live data. Client-side validation runs before every request: required fields, a Bilgi-email regex, a minimum password length, and password confirmation, each surfaced as an inline error near the relevant input.

Error handling (async safety). Every call is wrapped in `try/catch`; a failed response surfaces a friendly toast (“Server error.” / “Database connection failed.”) instead of breaking the UI:

```
try {
  const res = await fetch(url);
  if (!res.ok) throw new Error("Server Error");
} catch (e) {
  showToast("Network issue. Try again.");
}
```

Accessibility Review

Accessibility was treated as a core design requirement rather than an optional enhancement. The application targets WCAG 2.1 Level AA and supports inclusive use across diverse user groups.

WCAG principle	Implementation
Perceivable	High-contrast charcoal/white/red palette; readable type scale; status shown by text and color (Full / Available / Active / Pending / Unavailable) — never color alone
Operable	Full keyboard navigation; visible focus outlines; click targets at/above 44 px; persistent sidebar reduces memory load
Understandable	Clear labels, short corrective error messages near inputs, consistent layout across screens
Robust	Semantic HTML + ARIA: aria-live="polite" content/toast regions, role="dialog" + aria-modal on the confirmation modal, role="tablist"/role="tab" on facility filters, <label> on all fields

Additional measures: minimum 4.5:1 contrast target; text resizing support; logical focus order; error feedback announced to assistive technologies via live regions; inputs preserve user values on error; simple, step-by-step task flows that minimize cognitive load.

Final Reflection

The biggest challenge of this project was connecting the visual interface to system thinking. BilgiFit began as a set of static screens and grew into a working reservation system backed by a real database and an asynchronous API. Building the data model forced us to think about capacity rules, duplicate prevention, and what “cancel” really means (a status change, not a deletion), and those decisions then shaped the UI — disabled buttons, capacity bars, and confirmation modals all exist because the data model and business rules required them.

Designing the information architecture taught us to organize content by user goal so members can reserve a facility in a few predictable steps. We became more aware of cognitive load, consistency, and recognition-over-recall: small choices in labeling, button placement, and grouping made a large difference in how easy the system felt to use. The accessibility and validation work showed that good UI is not only visual polish — clear labels, keyboard support, and corrective error messages directly affect whether the system is usable for everyone.

On the technical side, mapping each UI action to a CRUD operation, a relational schema, and an asynchronous API made the connection between interface design and system architecture concrete. We now understand how front-end decisions directly affect data integrity, validation logic, and backend implementation.

Overall, this project taught us to approach interface design as a structured engineering and design process rather than a purely visual task — from personas and journeys through data modeling to a deployed, working application. We feel more confident designing user-centered systems, documenting design decisions, and communicating requirements between design and implementation, and we understand how thoughtful UI/UX design directly improves system effectiveness and user satisfaction.